

République du Sénégal



MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE

DIRECTION GÉNÉRALE DE L'ENSEIGNEMENT SUPÉRIEUR



Rapport de travail sur Docker

Présenté par : **Ibrahima Khalilou Thiam**

Master 2 Génie Logiciel UDB

Matricule 003

À **M. Massamba LO**

Année académique 2025

Sommaire

1. [Préparation & Installation de Docker](#)
2. [1er conteneur : exécution, listing, inspect](#)
3. [Déployer Nginx \(ports et mapping\)](#)
4. [Images : pull, inspect, save/load, commit](#)
5. [Docker Hub : tag & push](#)
6. [Réseaux : bridge, custom network, ping](#)
7. [Volumes & montages d'hôte](#)
8. [Dockerfile, build args et multi-stage](#)
9. [Docker Compose : stack & hot-reload](#)
10. [Configuration daemon.json \(DNS, logs, storage\)](#)
11. [Nettoyage & bonnes pratiques](#)
12. [Conclusion](#)



Préparation & Installation de Docker (Ubuntu/Debian)

Nous commençons par préparer l'environnement afin d'installer Docker Engine, containerd et le plugin docker-compose. Les commandes suivantes s'appliquent à une distribution Debian/Ubuntu ; nous les exécutons dans cet ordre pour garantir une installation propre et reproductible.

```
# 1. Mise à jour et installation des paquets nécessaires
sudo apt update
sudo apt install -y ca-certificates curl gnupg lsb-release

# 2. Ajouter la clé GPG officielle Docker
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o /etc/

# 3. Ajouter le dépôt Docker pour Debian
echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/debian \
$(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# 4. Installer Docker Engine, CLI et Compose plugin
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

# 5. Démarrer et activer le service Docker
sudo systemctl start docker
sudo systemctl enable docker
sudo systemctl status docker --no-pager
```

Ce que nous faisons et pourquoi :

- Nous mettons à jour les paquets pour partir d'un système à jour et éviter des conflits.
- Nous importons la clé GPG de Docker pour garantir l'intégrité des paquets téléchargés.
- Nous ajoutons le dépôt officiel Docker pour obtenir une version stable et maintenue.
- Nous installons Docker Engine et le plugin compose afin d'utiliser **docker** et **docker compose** (nouvelle syntaxe recommandée).

Remarque : Sur Windows ou macOS, nous utilisons Docker Desktop ; la logique de base (images, conteneurs, volumes) reste identique dans le terminal.

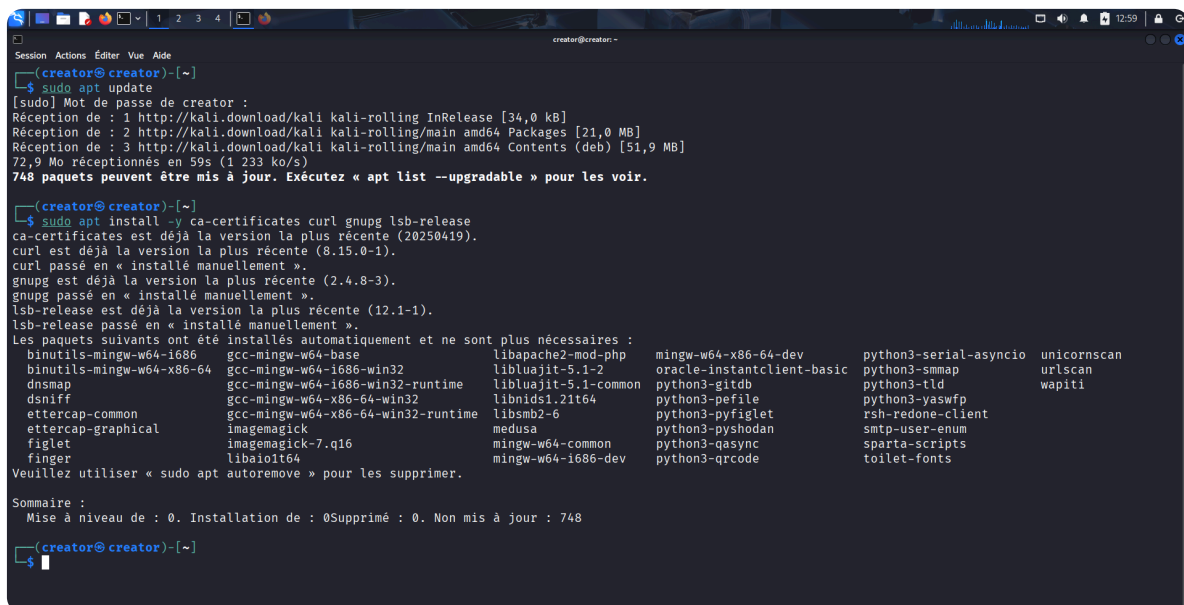
Choix de l'utilisateur qui exécute les conteneurs

Pour nos démonstrations en laboratoire nous choisissons souvent d'exécuter les conteneurs avec un utilisateur non root, mais pour simplifier les étapes et éviter des soucis de permissions lors du TP, nous identifions ici l'utilisateur qui exécutera les conteneurs. Dans cet exemple, nous l'ajoutons au groupe **docker** (ceci évite d'utiliser **sudo** pour chaque commande).

```
# Ajouter l'utilisateur courant au groupe docker
sudo usermod -aG docker $USER

# Pour appliquer la modification immédiatement sans déconnexion possible :
newgrp docker || echo "Déconnecte-toi et reconnecte-toi pour finaliser l'ajout au gr
```

Attention : l'ajout d'un utilisateur au groupe **docker** lui confère des capacités proches du root sur les ressources Docker — nous appliquons cette configuration uniquement sur des environnements de test ou machines personnelles.



```
Session Actions Éditer Vue Aide
(creator@creator)-[~]
$ sudo apt update
[sudo] Mot de passe de creator :
Réception de : 1 http://kali.download/kali kali-rolling InRelease [34,0 kB]
Réception de : 2 http://kali.download/kali kali-rolling/main amd64 Packages [21,0 MB]
Réception de : 3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [51,9 MB]
72,9 Mo réceptionnés en 59s (1 233 ko/s)
748 paquets peuvent être mis à jour. Exécutez « apt list --upgradable » pour les voir.

(creator@creator)-[~]
$ sudo apt install -y ca-certificates curl gnupg lsb-release
ca-certificates est déjà la version la plus récente (20250419).
curl est déjà la version la plus récente (8.15.0-1).
curl passé en « installé manuellement ».
gnupg est déjà la version la plus récente (2.4.8-3).
gnupg passé en « installé manuellement ».
lsb-release est déjà la version la plus récente (12.1-1).
lsb-release passé en « installé manuellement ».
Les paquets suivants ont été installés automatiquement et ne sont plus nécessaires :
binutils-mingw-w64-i686      gcc-mingw-w64-base          libapache2-mod-php          mingw-w64-x86-64-dev        python3-serial-asyncio      unicornscan
binutils-mingw-w64-x86-64    gcc-mingw-w64-i686-win32    liblua5.1-2                 oracle-instantclient-basic  python3-smmap               uulscan
dnsmasq                      gcc-mingw-w64-i686-win32-runtime  liblua5.1-common            python3-gitdb               python3-tld                 wapiti
dsniff                       gcc-mingw-w64-x86-64-win32      libnids1.21t64              python3-pefile              python3-yaswfp              rsh-redone-client          smtp-user-enum
ettercap-common              gcc-mingw-w64-x86-64-win32-runtime  libusb2-6                   python3-pyfiglet            python3-qasync              sparta-scripts             toilet-fonts
ettercap-graphical           imagemagick                   medusa                      python3-pyshodan            python3-qasync              sparta-scripts             toilet-fonts
figlet                       imagemagick-7.q16             mingw-w64-common            python3-qasync              python3-qrcode
finger                       libaio1t64                   mingw-w64-i686-dev          python3-qrcode
Veuillez utiliser « sudo apt autoremove » pour les supprimer.

Sommaire :
Mise à niveau de : 0. Installation de : 0Supprimé : 0. Non mis à jour : 748

(creator@creator)-[~]
$
```

```
Session Actions Éditer Vue Aide
748 paquets peuvent être mis à jour. Exécutez « apt list --upgradable » pour les voir.

(creator@creator)-[~]
$ sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
Les paquets suivants ont été installés automatiquement et ne sont plus nécessaires :
binutils-mingw-w64-i686 gcc-mingw-w64-base libapache2-mod-php mingw-w64-x86-64-dev python3-serial-asyncio unicornsanc
binutils-mingw-w64-x86-64 gcc-mingw-w64-1686-win32 liblua5.1-2 oracle-instantclient-basic python3-smmap urlscan
dnsmasq gcc-mingw-w64-1686-win32-runtime liblua5.1-common python3-github python3-tld wapiti
dsniff gcc-mingw-w64-x86-64-win32 libnids1.21t64 python3-pefile python3-yaswfp python3-redone-client
ettercap-common imagemagick libsm2-6 python3-pyfiglet python3-pyshodan smtp-user-enun
ettercap-graphical figlet imagemagick-7.q16 mingw-w64-common python3-qasync sparta-scripts
finger libaio1t64 mingw-w64-1686-dev python3-qrcode toilet-fonts

Veuillez utiliser « sudo apt autoremove » pour les supprimer.

Installation de :
containerd.io docker-ce docker-ce-cli docker-compose-plugin

Installation de dépendances :
docker-buildx-plugin docker-ce-rootless-extras libslirp0 pigz slirp4netns

Paquets suggérés :
cgrouperfs-mount | cgroup-lite docker-model-plugin

Sommaire :
Mise à niveau de : 0. Installation de : 9 Supprimé : 0. Non mis à jour : 748
Taille du téléchargement : 105 MB
Espace nécessaire : 436 MB / 8 904 MB disponible

Réception de : 4 https://download.docker.com/linux/debian bookworm/stable amd64 containerd.io amd64 1.7.28-0-debian.12-bookworm [31,9 MB]
Réception de : 1 http://kali.download.kali kali-rolling/main amd64 pigz amd64 2.8-1 [62,7 kB]
Réception de : 2 http://kali.download.kali kali-rolling/main amd64 libslirp0 amd64 4.9.1-1 [72,8 kB]
Réception de : 3 http://kali.download.kali kali-rolling/main amd64 slirp4netns amd64 1.3.3-1 [39,9 kB]
Réception de : 5 https://download.docker.com/linux/debian bookworm/stable amd64 docker-ce-cli amd64 5:28.5.1-1-debian.12-bookworm [16,5 MB]
Réception de : 6 https://download.docker.com/linux/debian bookworm/stable amd64 docker-ce amd64 5:28.5.1-1-debian.12-bookworm [19,7 MB]
48% [6 docker-ce 0 B/19,7 MB 0%] 2 758 kB/s 20s
```

```
Applications Sessions Éditer Vue Aide
$ sudo systemctl start docker

(creator@creator)-[~]
$ sudo systemctl enable docker
Synchronizing state of docker.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker

(creator@creator)-[~]
$ sudo systemctl status docker --no-pager
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Fri 2025-10-10 13:18:54 CEST; 2min 28s ago
 Invocation: 922f047650314fe19f3a8eef311d88cb
 TriggeredBy: ● docker.socket
   Main PID: 242023 (dockerd)
    Tasks: 8
   Memory: 23,8M (peak: 26M)
    CPU: 2,096s
   CGroup: /system.slice/docker.service
           └─242023 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

oct. 10 13:18:49 creator dockerd[242023]: time="2025-10-10T13:18:49.592884474+02:00" level=info msg="CDI directory does not exist, skipping: fail_var/run/cdi"
oct. 10 13:18:49 creator dockerd[242023]: time="2025-10-10T13:18:49.818136620+02:00" level=info msg="Creating a containerd client" address=/run/c...imeout=1m0s
oct. 10 13:18:50 creator dockerd[242023]: time="2025-10-10T13:18:50.357492374+02:00" level=info msg="Loading containers: start."
oct. 10 13:18:53 creator dockerd[242023]: time="2025-10-10T13:18:53.356335542+02:00" level=info msg="Loading containers: done."
oct. 10 13:18:53 creator dockerd[242023]: time="2025-10-10T13:18:53.740476856+02:00" level=info msg="Docker daemon" commit=f8215cc containerd-sna...sion=28.5.1
oct. 10 13:18:53 creator dockerd[242023]: time="2025-10-10T13:18:53.747320566+02:00" level=info msg="Initializing buildkit"
oct. 10 13:18:54 creator dockerd[242023]: time="2025-10-10T13:18:54.034151096+02:00" level=info msg="Completed buildkit initialization"
oct. 10 13:18:54 creator dockerd[242023]: time="2025-10-10T13:18:54.044676278+02:00" level=info msg="Daemon has completed initialization"
oct. 10 13:18:54 creator dockerd[242023]: time="2025-10-10T13:18:54.045036324+02:00" level=info msg="API listen on /run/docker.sock"
oct. 10 13:18:54 creator systemd[1]: Started docker.service - Docker Application Container Engine.
Hint: Some lines were ellipsized, use -l to show in full.

(creator@creator)-[~]
$
```

```
Applications Sessions Éditer Vue Aide
(creator@creator)-[~]
$ sudo usermod -aG docker $USER

(creator@creator)-[~]
$ newgrp docker || echo "Déconnecte-toi et reconnecte-toi pour finaliser l'ajout au groupe"
(creator@creator)-[~]
$
```

Lancer notre premier conteneur, lister, inspecter, entrer

Nous lançons un conteneur Debian interactif pour vérifier l'environnement d'exécution et nous familiariser avec les commandes de base : démarrage, listing et inspection des métadonnées.

```
# Lancer un conteneur debian en mode interactif (bash), puis sortir avec 'exit'
docker run -it --name test-debian debian:latest bash

# Lister les conteneurs en cours (running)
docker ps

# Lister tous les conteneurs (y compris stopped)
docker ps -a

# Inspecter un conteneur (métadonnées JSON)
docker inspect test-debian

# Exécuter une commande dans un conteneur déjà démarré (ex: ouvrir un shell)
docker exec -it <container_id_or_name> bash
```

Observations :

- Avec `docker inspect` nous obtenons des informations précises (IP, volumes, commandes d'entrée) utiles pour le diagnostic.

Astuce : Nous pouvons utiliser `--rm` pour que le conteneur soit automatiquement supprimé après arrêt, ce qui évite l'accumulation de conteneurs tests.

```
Éditeur de texte
Éditeur de texte simple Aide

creator@creator: ~
$ docker run -it --name test-debian debian:latest bash

Unable to find image 'debian:latest' locally
latest: Pulling from library/debian
cae3b572364a: Pull complete
Digest: sha256:fd8f5a1df07b5195613e4b9a0b6a947d3772a151b81975db27d47f093f60c6e6
Status: Downloaded newer image for debian:latest
root@1ced78354f7e:/# exit
exit

creator@creator: ~
$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES

creator@creator: ~
$ docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED      STATUS      PORTS   NAMES
1ced78354f7e   debian:latest   "bash"    2 minutes ago   Exited (0) 26 seconds ago   test-debian

creator@creator: ~
$ docker inspect test-debian
[
  {
    "Id": "1ced78354f7e8256f7a44e3d2f9c997cdc2b76af237e31bcfe58333f139f4547",
    "Created": "2025-10-10T11:49:00.735765245Z",
    "Path": "bash",
    "Args": [],
    "State": {
      "Status": "exited",
      "Running": false,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
```

Déployer Nginx — exposition de ports et accès depuis le navigateur

Nous déployons un serveur Nginx dans un conteneur afin de comprendre le mapping de ports entre l'hôte et le conteneur, puis nous accédons à la page par le navigateur pour valider le déploiement.

```
# Déployer Nginx en détaché et exposer le port 80 du conteneur sur 8090 de l'hôte
docker run -d --name web-nginx -p 8090:80 nginx:stable

# Vérifier le conteneur
docker ps

# Accès depuis navigateur : http://<IP_de_la_machine>:8090
# Si tu es en local (même machine), utilise : http://localhost:8090
```

Explication pratique :

Le paramètre `-p 8090:80` signifie que nous redirigeons le port 8090 de la machine hôte vers le port 80 du conteneur. Sans option `-p`, le service reste isolé et n'est pas accessible directement depuis l'extérieur.

Exécution sans préciser l'hôte (port aléatoire)

```
# Exécuter sans -p (Docker alloue un port aléatoire côté hôte si demandé)
docker run -d --name web3 -P nginx:stable

# Voir le mapping
docker port web3
```

```
Session Actions Éditer Vue Aide

creator@creator: ~
$ docker run -d --name web-nginx -p 8090:80 nginx:stable
Unable to find image 'nginx:stable' locally
stable: Pulling from library/nginx
5c32499ab806: Pull complete
ae7b49ada9e3: Pull complete
8e5924dfa87c: Pull complete
1d9a18bc0c05: Pull complete
be90cf255959: Pull complete
82eb62151b9d: Pull complete
08d4f638eff8: Pull complete
Digest: sha256:3b4019335070fb6445987c8dd72cf18dec2cbc63b3575581eb66469c8173cd4f
Status: Downloaded newer image for nginx:stable
0cedeb61263ca231edbb7bf5122d6c117305e0aa55d92b56ea198f51eb97c623

creator@creator: ~
$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
0cedeb61263c   nginx:stable  "/docker-entrypoint..."  20 seconds ago Up 18 seconds  0.0.0.0:8090→80/tcp, [::]:8090→80/tcp  web-nginx

creator@creator: ~
$
```

```
Session Actions Éditer Vue Aide

creator@creator: ~
$ docker run -d --name web-nginx -p 8090:80 nginx:stable
Unable to find image 'nginx:stable' locally
stable: Pulling from library/nginx
5c32499ab806: Pull complete
ae7b49ada9e3: Pull complete
8e5924dfa87c: Pull complete
1d9a18bc0c05: Pull complete
be90cf255959: Pull complete
82eb62151b9d: Pull complete
08d4f638eff8: Pull complete
Digest: sha256:3b4019335070fb6445987c8dd72cf18dec2cbc63b3575581eb66469c8173cd4f
Status: Downloaded newer image for nginx:stable
0cedeb61263ca231edbb7bf5122d6c117305e0aa55d92b56ea198f51eb97c623

creator@creator: ~
$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
0cedeb61263c   nginx:stable  "/docker-entrypoint..."  20 seconds ago Up 18 seconds  0.0.0.0:8090→80/tcp, [::]:8090→80/tcp  web-nginx

creator@creator: ~
$
```

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

Images : récupérer, inspecter, sauvegarder et recréer

Nous manipulons des images Docker (pull, inspect, save/load) et démontrons comment transformer un conteneur modifié en image pour réutilisation ou partage.

```
# Pull d'une image Alpine
docker pull alpine:latest

# Lister les images (affichage standard)
docker images

# Lister images sans troncature d'ID
docker images --no-trunc

# Filtrer les images par nom
docker images --filter=reference='alpine*'

# Inspecter une image (métadonnées)
docker image inspect alpine:latest

# Faire un commit d'un conteneur en image (créer une image à partir d'un conteneur modifié)
docker commit <container_id_or_name> monrepo/monimage:tag

# Sauvegarder une image dans un tar
docker save -o nginx-image.tar nginx:stable

# Charger une image depuis un tar
docker load -i nginx-image.tar
```

Commentaires :

- Nous utilisons `docker commit` pour capturer rapidement l'état d'un conteneur lors d'un test, mais nous privilégions l'écriture d'un `Dockerfile` pour la reproductibilité.
- `docker save/load` est pratique pour transporter des images entre environnements isolés ou pour archivage.

```
Session Actions Éditer Vue Aide
]

(creator@creator)-[~]
$ docker pull alpine:latest
latest: Pulling from library/alpine
2d35ebdb57d9: Pull complete
Digest: sha256:4b7ce07002c69e8f3d704a9c5d6fd3053be500b7f1c69fc0d80990c2ad8dd412
Status: Downloaded newer image for alpine:latest
docker.io/library/alpine:latest

(creator@creator)-[~]
$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
alpine        latest    706db57fb206   2 days ago     8.32MB
debian        latest    61d0976aceca   11 days ago    120MB
nginx         stable    bf0a563e3e6a   8 weeks ago    192MB

(creator@creator)-[~]
$ docker images --no-trunc
REPOSITORY    TAG       IMAGE ID                                               CREATED        SIZE
alpine        latest    sha256:706db57fb2063f39f69632c5b5c9c439633fda35110e65587c5d85553fd1cc38   2 days ago     8.32MB
debian        latest    sha256:61d0976aceca49cd8c5a3fb6787ddb08bd43a8f998fef45bf3d1050420161be7     11 days ago    120MB
nginx         stable    sha256:bf0a563e3e6a4c41dcdb46a5c1db44dd74662631dd2d360b5a3e9f5ebdecaF27     8 weeks ago    192MB

(creator@creator)-[~]
$ docker images --filter=reference='alpine*'
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
alpine        latest    706db57fb206   2 days ago     8.32MB

(creator@creator)-[~]
$ docker image inspect alpine:latest
[
  {
    "Id": "sha256:706db57fb2063f39f69632c5b5c9c439633fda35110e65587c5d85553fd1cc38",
    "RepoTags": [
      "alpine:latest"
    ],
  }
]
```

```
Session Actions Éditer Vue Aide
]

(creator@creator)-[~]
$ docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
0cedeb61263c   nginx:stable   "/docker-entrypoint..." 10 minutes ago Up 10 minutes   0.0.0.0:8090->80/tcp, [::]:8090->80/tcp   web-nginx
1ced78354f7e   debian:latest   "bash"                    16 minutes ago Exited (0) 14 minutes ago               test-debian

(creator@creator)-[~]
$ docker commit 1ced78354f7e monrepo/monimage:tag
sha256:bfd7a696e7ad874a0be93908145f0c9f28f9f1e49f91ad51eec1e81cdcd72c

(creator@creator)-[~]
$ docker save -o nginx-image.tar nginx:stable

(creator@creator)-[~]
$ docker load -i nginx-image.tar
Loaded image: nginx:stable

(creator@creator)-[~]
$

(creator@creator)-[~]
$

(creator@creator)-[~]
$

(creator@creator)-[~]
$

(creator@creator)-[~]
$

(creator@creator)-[~]
$
```

Docker Hub — afficher, tagger et pousser une image

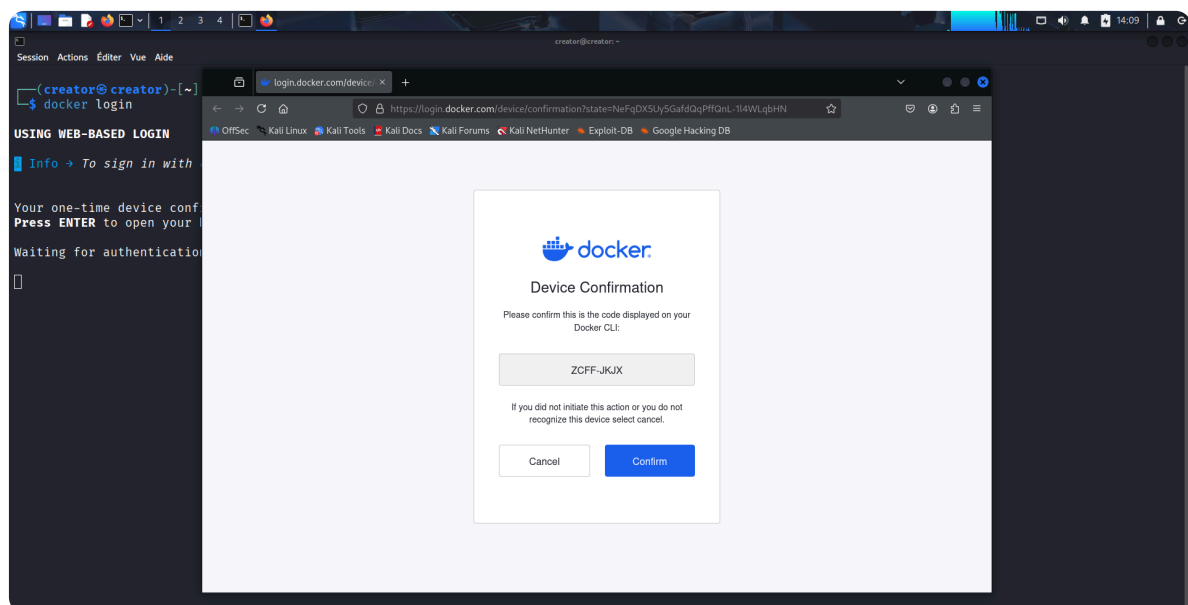
Nous publions une image sur Docker Hub pour la partager. Cette étape nécessite un compte et une authentification préalables.

```
# Se connecter au Docker Hub
docker login

# Tagger l'image locale avec le préfixe Docker Hub (username/image:tag)
docker tag monrepo/monimage:tag tonuser/monimage:tag

# Pousser
docker push tonuser/monimage:tag
```

Remarque : une fois poussée, nous vérifions la présence de l'image sur Docker Hub via l'interface web pour s'assurer que le push s'est bien déroulé.



```
Session Actions Éditer Vue Aide
creator@creator: ~

$ docker login

USING WEB-BASED LOGIN

Info → To sign in with credentials on the command line, use 'docker login -u <username>'

Your one-time device confirmation code is: ZCFF-JKJX
Press ENTER to open your browser or submit your device code here: https://login.docker.com/activate
Waiting for authentication in the browser..

WARNING! Your credentials are stored unencrypted in '/home/creator/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded

$ docker tag monrepo/monimage:tag tonuser/monimage:tag

$ docker push tonuser/monimage:tag
The push refers to repository [docker.io/tonuser/monimage]
3c42597b60ff: Preparing
a5ec5ec9d16c: Preparing
denied: requested access to the resource is denied

$
```

Réseaux Docker — bridge par défaut et réseau personnalisé

Nous explorons le réseau bridge par défaut, créons un réseau pont personnalisé et testons la communication entre conteneurs en utilisant le DNS interne fourni par Docker.

```
# Lister les réseaux
docker network ls

# Inspecter le réseau bridge par défaut
docker network inspect bridge

# Créer un réseau bridge personnalisé
docker network create --driver bridge mynet

# Lancer deux conteneurs et les connecter au réseau personnalisé
docker run -d --name cont1 --network mynet alpine sleep 3600
docker run -d --name cont2 --network mynet alpine sleep 3600

# Entrer dans cont1 et ping cont2 (installer iputils si nécessaire)
docker exec -it cont1 sh
# Dans le conteneur : apk add --no-cache iputils && ping -c 3 cont2
```

Observations réseau :

- Les conteneurs connectés au même réseau peuvent se joindre par nom grâce au DNS interne de Docker.
- Pour tester la connectivité, nous installons les utilitaires nécessaires (ex : ping).

Test d'accès Internet : depuis un conteneur nous exécutons `apk update` puis `ping -c 3 8.8.8.8` pour valider l'accès externe.

```
Session Actions Éditer Vue Aide creator@creator: ~

$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
65db2c4b41b6        bridge             bridge             local
e87695da13bb        host               host               local
f66fdfed0700        none              null               local

$ docker network inspect bridge
[
  {
    "Name": "bridge",
    "Id": "65db2c4b41b6896070f7583b92071cb8ade542c2b5e110fbf856e9fa96737592",
    "Created": "2025-10-10T13:18:52.499016781+02:00",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "172.17.0.0/16",
          "Gateway": "172.17.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
```

Volumes & montages d'hôte — persistance des données

Nous démontrons deux approches pour la persistance : les bind mounts (montage d'un dossier hôte) et les volumes Docker (gérés par Docker). Nous vérifions que les données persistent entre conteneurs.

```
# 1) Bind mount (dossier hôte monté dans le conteneur)
mkdir -p /home/$USER/docker-shared
docker run -d --name volsharing1 -v /home/$USER/docker-shared:/var/www/html -p 8081:80

# 2) Volume Docker (géré par Docker)
docker volume create mydata
docker run -d --name busybox-cont -v mydata:/data busybox sh -c "sleep 3600"

# Dans le conteneur -> créer un fichier
docker exec -it busybox-cont sh -c "echo 'hello from container' > /data/hello.txt"

# Monter un autre conteneur sur le même volume et lire le fichier
docker run --rm -it -v mydata:/data alpine cat /data/hello.txt
```

Pourquoi choisir l'un ou l'autre :

- Les bind mounts reflètent exactement un répertoire de l'hôte — pratique pour le développement (live-edit, hot-reload).
- Les volumes Docker sont isolés de l'hôte et plus portables, ce qui les rend recommandés en environnement de production.
- Nous utilisons ces exemples pour montrer la persistance et le partage de fichiers entre conteneurs.

```
Applications  ons  Éditer  Vue  Aide  creator@creator: ~  
[creator@creator]~  
$ mkdir -p /home/$USER/docker-shared  
[creator@creator]~  
$ docker run -d --name volsharing1 -v /home/$USER/docker-shared:/var/www/html -p 8081:80 nginx:stable  
c90915022a459ce5e0235d2df863392834348c7f4436cd180868055d11b5844b  
[creator@creator]~  
$ docker volume create mydata  
mydata  
[creator@creator]~  
$ docker run -d --name busybox-cont -v mydata:/data busybox sh -c "sleep 3600"  
Unable to find image 'busybox:latest' locally  
latest: Pulling from library/busybox  
80bfb88a41a2: Pull complete  
Digest: sha256:d82f458899c9696cb26a7c02d5568f81c8c8223f8661bb2a7988b269c8b9051e  
Status: Downloaded newer image for busybox:latest  
3209018c77d0501e1abd7c123ffa9493ceb684f6ec7fc36a2b6cbcdc51ab3563  
[creator@creator]~  
$ docker exec -it busybox-cont sh -c "echo 'hello from container' > /data/hello.txt"  
[creator@creator]~  
$ docker run --rm -it -v mydata:/data alpine cat /data/hello.txt  
hello from container  
[creator@creator]~  
$
```

Dockerfile, ARG, ENV et multi-stage builds

Nous écrivons des Dockerfiles reproductibles, expliquons la différence entre **ARG** et **ENV**, et montrons un exemple multi-stage pour réduire la taille de l'image finale.

Exemple simple — Dockerfile (nginx personnalisé)

```
# Dockerfile
FROM nginx:stable
COPY index.nginx-debian.html /usr/share/nginx/html/index.nginx-debian.html
COPY start.sh /usr/local/bin/start.sh
RUN chmod +x /usr/local/bin/start.sh
codeblock ["/usr/local/bin/start.sh"]
```

```
# start.sh (script de démarrage)
#!/bin/sh
# script simple qui démarre nginx en foreground
nginx -g 'daemon off;'
```

Build & run :

```
docker build -t mon-nginx:custom .
docker run -d --name mon-nginx -p 8082:80 mon-nginx:custom
```

ARG vs ENV — explication

```
# Exemple - Dockerfile snippet
ARG USERNAME=some_user
ENV RUN_USER=${USERNAME}
# ARG est utilisé uniquement au build-time ; ENV est conservée dans l'image et dispon
```

Exemple multi-stage (Node app)

```
# Multi-stage Dockerfile (extrait)
# Stage 1 : build
FROM node:20 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install --production=false
COPY . .
RUN npm run build

# Stage 2 : runtime léger
FROM node:20-slim
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY package*.json ./
RUN npm install --production=true
codeblock ["node", "dist/server.js"]
```

Pourquoi le multi-stage :

Nous séparons l'étape de compilation contenant tous les outils nécessaires du runtime qui doit être minimal. Ainsi, l'image finale est plus légère et plus sécurisée.

Docker Compose — déployer une stack et activer le hot-reload

Nous déployons une application multi-conteneurs avec Docker Compose et utilisons des volumes pour le développement afin d'obtenir un rechargement à chaud des fichiers sources.

```
# docker-compose.yml (exemple basique)
version: '3.8'
services:
  web:
    build: ./app
    ports:
      - "8083:80"
    volumes:
      - ./app:/usr/share/nginx/html:ro
  redis:
    image: redis:7
```

```
# Lancer la stack
docker compose up -d --build

# Vérifier les services
docker compose ps

# Voir les logs
docker compose logs -f web
```

Remarque :

En montant le dossier source de l'hôte dans le conteneur (**volumes:** **./app:/usr/share/nginx/html**), nos modifications sont prises en compte immédiatement sans rebuild — très utile en développement.

Configuration daemon.json — DNS, logger et storage driver

Nous modifions la configuration du démon Docker via `/etc/docker/daemon.json` pour personnaliser le comportement réseau, de logs et de stockage selon nos besoins.

```
# Exemple /etc/docker/daemon.json
{
  "dns": ["8.8.8.8", "1.1.1.1"],
  "log-driver": "json-file",
  "storage-driver": "overlay2"
}
# Après modification, redémarrer docker
sudo systemctl restart docker
```

Points importants :

- Modifier `dns` est utile si nos conteneurs ne résolvent pas correctement des noms.
- Le `log-driver` influe sur la gestion et la rotation des logs — à adapter selon la taille de l'infra.
- Le `storage-driver` recommandé pour la plupart des systèmes modernes est `overlay2`.

Nettoyage & bonnes pratiques

Pour garder un environnement propre, nous montrons les commandes utiles pour arrêter et supprimer des conteneurs, images et volumes non utilisés. Nous rappelons toutefois la prudence en production.

```
# Arrêter et supprimer tous les conteneurs (à utiliser avec prudence)
docker stop $(docker ps -aq)
docker rm $(docker ps -aq)

# Supprimer images non utilisées
docker image prune -a

# Supprimer volumes non utilisés
docker volume prune

# Nettoyage complet (dangereux : supprime tout ce qui n'est pas utilisé)
docker system prune -a --volumes
```

Important : ces commandes suppriment des données — nous vérifions toujours l'impact avant exécution, surtout sur des machines partagées ou en production.

version

```
# Vérifier docker
docker --version
sudo systemctl status docker

# Stop / remove conteneurs
docker stop $(docker ps -aq)
docker rm $(docker ps -aq)

# Liste images (taille)
docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}"
```

```
Reduire toutes les fenetres ouvertes et afficher le bureau
creator@creator: ~
$ docker --version
28.5.1
$ sudo systemctl status docker
# Stop / remove conteneurs
docker stop $(docker ps -aq)
docker rm $(docker ps -aq)

# Liste images (taille)
docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}"
Docker version 28.5.1, build e180ab8
[sudo] Mot de passe de creator :
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Fri 2025-10-10 13:18:54 CEST; 1h 12min ago
 Invocation: 922f047650314fe19f3a8eef311d88cb
  TriggeredBy: ● docker.socket
    Main PID: 242023 (dockerd)
      Tasks: 35
     Memory: 89.9M (peak: 487.4M, swap: 3.4M, swap peak: 3.4M)
        CPU: 1min 25.389s
    CGroup: /system.slice/docker.service
            └─242023 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
            └─260014 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 8090 -container-ip 172.17.0.2 -container-port 80 -use-listen-fd
            └─260018 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 8090 -container-ip 172.17.0.2 -container-port 80 -use-listen-fd
            └─277324 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 8081 -container-ip 172.17.0.3 -container-port 80 -use-listen-fd
            └─277329 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 8081 -container-ip 172.17.0.3 -container-port 80 -use-listen-fd

oct. 10 13:18:54 creator dockerd[242023]: time="2025-10-10T13:18:54.045036324+02:00" level=info msg="API listen on /run/docker.sock"
oct. 10 13:18:54 creator systemd[1]: Started docker.service - Docker Application Container Engine.
oct. 10 13:30:12 creator dockerd[242023]: time="2025-10-10T13:30:12.780370337+02:00" level=error msg="Not continuing with pull after error" error="manifest
oct. 10 13:34:36 creator dockerd[242023]: time="2025-10-10T13:34:36.361586181+02:00" level=info msg="ignoring event" container=a640f76322a488ce9a17dd6727dfd
oct. 10 13:44:30 creator dockerd[242023]: time="2025-10-10T13:44:30.190260072+02:00" level=info msg="ignoring event" container=36e15ed855c1b9eb85aed8a8a60eb
oct. 10 13:47:43 creator dockerd[242023]: time="2025-10-10T13:47:43.681889086+02:00" level=info msg="ignoring event" container=fc83c190657fa445ed2314e1bfb79
```

Conclusion

Ce travail pratique a permis d'acquérir une maîtrise progressive de Docker, depuis l'installation initiale jusqu'au déploiement de stacks multi-conteneurs. Nous avons pu :

- Installer et configurer Docker sur Kali Linux, en vérifiant le bon fonctionnement des services et des versions.
- Créer, exécuter et gérer des conteneurs pour différentes images (Debian, Nginx), tout en testant l'exposition des ports et la communication réseau.
- Manipuler les images : pull, push vers Docker Hub, sauvegarde, chargement et gestion des tags.
- Mettre en place des volumes pour la persistance des données et utiliser Docker Compose pour orchestrer plusieurs services.
- Construire des images avec Dockerfile, comparer single-stage et multi-stage, et appliquer des optimisations pour réduire la taille des images.
- Résoudre des problèmes pratiques rencontrés, tels que les conflits de ports, la configuration DNS dans les conteneurs ou l'authentification Docker Hub.

Ces étapes ont permis de consolider les connaissances théoriques par la pratique et de comprendre les bonnes pratiques à adopter : gestion des volumes, sécurité des secrets, configuration des logs, et utilisation d'une CI pour des builds reproductibles.

En résumé, ce TP illustre l'ensemble du cycle de vie d'un conteneur Docker, depuis la configuration de l'environnement jusqu'au déploiement d'applications orchestrées, offrant une base solide pour des projets futurs en développement et en administration de systèmes conteneurisés.

Ressources & liens rapides

- [Docker Docs](#)
- [Docker Hub](#)
- [Commande Docker \(référence\)](#)

Récap commandes essentielles

- `docker run`
- `docker ps`, `docker ps -a`
- `docker images`, `docker pull`
- `docker build`, `docker push`
- `docker-compose up -d` / `docker compose`